

BonusGameWorld01

A Sample Project Mini-Cookbook

This booklet is designed to accompany the sample project `BonusgameWorld01` from the book [Practical C++ Game Programming with Data Structures and Algorithms](#). It provides additional insights into the code logic, algorithms, and problem-solving approaches used in the implementation of this sample.

In this mini-booklet, you'll find how the sample project works and the key ideas behind its design.

Important note

If you purchased the book after **September 30, 2025**, the content of this mini-cookbook is already included in the *Integrating features into a cohesive game world* section of *Chapter 11*, so you do not need to read this mini-cookbook separately.

Before we begin

You will need the knowledge of several chapters to understand how the sample works. Make sure you've finished reading till the end of *Chapter 9*. It's the best time to try out this sample before moving to *Chapter 10*. This booklet will not repeatedly cover topics you've learned till *Chapter 9*.

Integrating features into a cohesive game world

From *Chapter 1* to *Chapter 10* of our book, we focused on introducing various features commonly used in game development. In those chapters, each demo project was designed specifically to demonstrate how to implement an individual feature. Now it's time to bring all those features together.

Chances are, as you've been reading the book, you've already started building your own game project and experimenting with integrating the features we introduced into your own work.

During this process, you may have noticed something important: combining multiple features into a single project often requires additional hidden effort and problem-solving.

The sample project `BonusGameWorld01`, shown in *Figure 1*, was created with that exact purpose in mind. It integrates many of the features we covered in earlier chapters to build a game scene similar to an open-world game. This includes a scene where the player can freely move around and explore, along with other decorative objects, and visual effects that enrich the environment.

Getting the project up and running

First, make sure you have pulled the latest updates from GitHub:

<https://github.com/PacktPublishing/Practical-C-Game-Programming-with-Data-Structures-and-Algorithms/tree/main>.

Open the Visual Studio solution file included with this book in Visual Studio Community, select the `BonusGameWorld01` project, rebuild it, and run. You should see the following similar output. You can use *WASD* to move the player character, rotate the camera with the mouse while holding the *right button*, or use the *space bar* to fire attacking magic.



Figure 1 – The BonusGameWorld01 sample project

This project differs from the other samples in the book. While the other samples focus on explaining and demonstrating a single feature at a time, this sample project attempts to integrate multiple features into one cohesive experience. It incorporates the following features we previously introduced and implemented:

The Knight application architecture

The overall project follows the same Knight application architecture introduced in *Chapters 1* and *2*, with `Entity` classes handling gameplay logic and using the scene graph components (`SceneActor` class) to handle rendering of objects.

The demo only has 4 Entities. The `TerrainEntity` handles not only the Quad-tree terrain, but also an additional 600 decorative `SceneActor` objects (trees and other props) on the terrain.

Simulation of daylight changes

The color blending technique introduced in *Chapter 4* is used in class to simulate daylight changes in the `TerrainEntity` class. We use the blended color as the diffuse color of the scene directional light and set it in the `Update()` function of `BonusGameWorld01` main application class.

```
//change of sky light
```

```
sceneLight->lightColor = pMainCamera->  
GetComponent<SkyboxComponent>()->_SkyColor;
```

Billboards of trees and other props

We chose to use the Billboard technique to render lots of trees and other props in the scene. The `BillboardComponent` class is explained in *Chapter 7*.

Third-person view camera control

We chose the third-person follow-up camera to render the game scene from behind the player character. This camera also allows the player to further rotate around to get the surrounding views. We use the `FollowUpCamera` class introduced in *Chapter 5*.

Lighting and shadow

A directional light is used to simulate sky light. It's also used as a primary light to cast shadows. Many objects, including Billboard-rendered trees and props, cast and receive shadows. We use the same `DepthRenderPass` class and `ShadowRenderPass` class introduced in *Chapters 6* and *7* to handle multi-pass rendering tasks.

Attacking and other scene effects

The magic fountain and player character's magic attack effects are done with the particle system introduced in *Chapter 7*. In this project, we demonstrate the use of multiple independent particle systems at the same time. The particles will keep live on for the magic fountains in the scene, while the particle streams for the player's attacks die out quickly with each strike.

Open world outdoor terrain and SkyBox

We use the same height map with the Quadtree terrain rendering algorithm introduced in *Chapter 7*.

We use the `SkyboxComponent` class introduced in *Chapter 7* to handle seamless sky rendering. It's attached under the camera `SceneActor`, so it always moves with the camera and appears infinitely far from the viewpoint.

Character model and animation states

The control of player model animation uses the finite state machine introduced in *Chapter 9*. The states are tailored for this sample project in the `PlayerFSM` class under `BonusGameWorld01` project folder.

Practical adjustments for integration

To assemble all the existing components to work together, it also takes more effort than just ripping the code from each individual sample project.

Adjusting shadow rendering properties

In the sample code of `Demo7PCFShadow`, the `ShadowSceneLight` orthogonal projection size is set to a constant value of 20 in the `Create()` function of the `DepthRenderPass` class. This configuration restricts shadow rendering to a relatively small, focused area. In contrast, the third-person view camera in this example covers a much larger portion of the terrain.

As a result, all shadows cast onto any objects within the camera's view must be rendered. To accommodate this larger region, the orthogonal projection scale is increased to 100 in this sample. This is done in the `Create()` function of the `DepthRenderPass` class of this sample project:

```
// Orthographic projection for directional light
lightCam.projection = CAMERA_ORTHOGRAPHIC;
lightCam.fovy = 100.0f;
//...
```

You can try to tweak this value to see how it affect the shadow quality.

In your own project, you may need to adjust this value to achieve the best visual quality.

If the orthogonal projection scale is set too large, shadows will appear overly blurred. Conversely, if the scale is set too small, parts of the shadows may be clipped by the boundaries of the shadow map texture, resulting in visually incorrect artifacts.

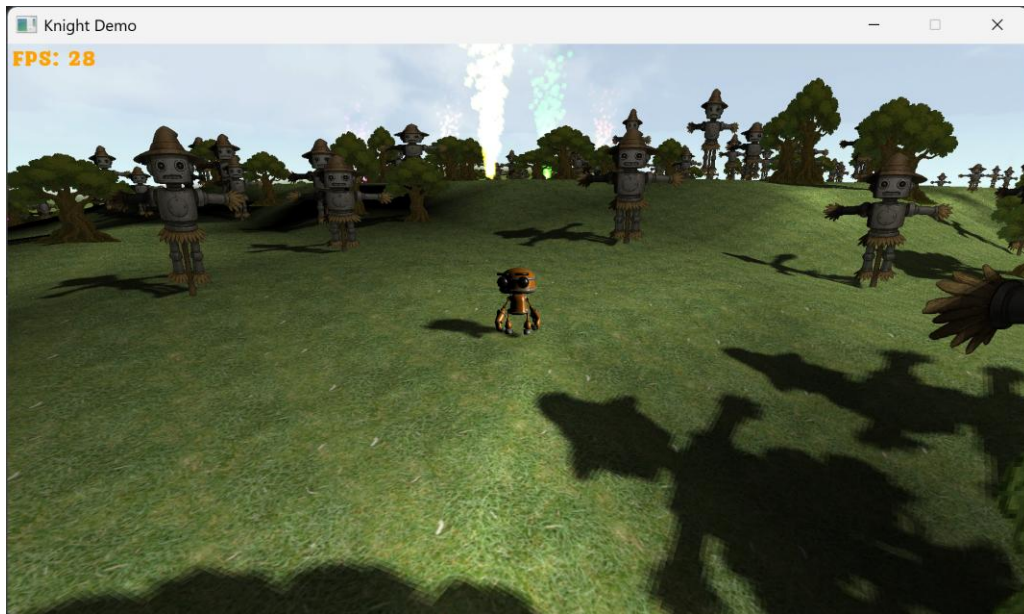


Figure 2 - Adjusting the shadow camera to achieve acceptable shadow quality

Placing objects on the terrain

To enable other components to query the elevation of a specific world-space position, we implemented an additional function, `GetTerrainY()`, in the `QuadTreeTerrainComponent` class. This function also takes into consideration the scale of the terrain.

The `TerrainEntity` class relies on this function to align all trees and other props with the terrain elevation in world space.

Moving the player character and camera

The character movement also depends on `GetTerrainY()` to update the player character's position, ensuring the character properly follows the terrain's contours.

The third-person follow-up camera must adjust its height (Y-axis position in world space) according to the terrain elevation at its current world position. This ensures that the camera does not move beneath the terrain surface, which would otherwise result in visual artifacts.

Additionally, the vertical offset between the player character and the camera should not be rigidly constrained. Enforcing such a lock can cause noticeable jitters in vertical movement when the player moves on very steep slopes. This implementation detail is illustrated in the `Update()` function of the main application class, `BonusGameWorld01`.

Simplifying the calculation

For simplicity, we create almost all `SceneActor` graphic objects at the top level of the scene graph. This makes them the direct descendants of the `SceneRoot` node, and their world positions are the same as their local positions. In this way, we can simply assign the local position of a `SceneActor` to make it move to the same world position.

If you put a player `SceneActor` as a child of another `SceneActor`, you would need to update its local position and then call `GetWorldPosition()` function in the `SceneActor` class to get the final world position.

If you need to move a `SceneActor` to a specified world position, you must reverse calculate and update the local position in the `SceneActor` after you move it. You can check what needs to be done in the handy `TranslateWS()` function in the `SceneActor` class of Knight.

This simplifies the calculation of object transformation in the world space, and is enough for the current demonstration purpose.

Conclusion

Through this sample, we want to illustrate the kinds of issues and programming considerations you may encounter when combining individual features into a complete project.

Challenge yourself

You may have noticed that once we place terrain, the main character, and many decorative objects into the scene, the rendering performance drops significantly. In the

FREE mini-cookbook for sample project from Practical C++ Games Data Structure and Algorithms

next demo, `BonusGameWorld02`, we will introduce techniques for optimizing rendering performance and better refactoring the code for this project.

Happy coding and learning!